

CMU_DSCLUB_Spot_the_Mask_with_FastAI_DH

February 5, 2026

1 Starter Notebook - Spot The mask

This notebook is going to run through all the steps in creating a submission for the [SPOT THE MASK contest](#) on Zindi. If you want to skip the explanations, scroll down to the end for a minimal example submission from scratch in 10 lines of code.

2 High level Understanding.

For this challenge, we were given a train dataset that contains 1800+ images of people wearing mask or not. The objective of this challenge is to create an image classification machine learning model to accurately predict the likelihood that an image contains a person wearing a face mask, or not

The stakeholders believe your machine learning solution will help policymakers, law enforcement, hospitals, and even commercial businesses ensure that masks are being worn appropriately in public, and that The solutions can help in the battle to reduce community transmission of COVID-19.

This tutorial will give you some insight on how it is done.

3 Setup

Before we can start modelling, we need to get the data somehow. Since this example is running in Colab, you have several options: - Upload everything through the 'Files' tab on the left - Upload the files to your Google Drive and then use the 'Mount Drive' option to oad your drive like and external hard drive. - Download the files directly from Kaggle [here](#) or better still use the kaggle library to get it in smoothly.

I've uploaded the two CSV files manually, since they're small. The images are only ~200MB zipped, but why waste your home internet? You can then use the line of code below to download the file:

```
[ ]: import os
import kagglehub

# Download latest version
path = kagglehub.dataset_download("victorolufemi/spot-the-mask-challenge")

print("Path to dataset files:", path)
```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.12), please consider upgrading to the latest version (0.4.1).

Downloading from

https://www.kaggle.com/api/v1/datasets/download/victorolufemi/spot-the-mask-challenge?dataset_version_number=1...

100%| | 192M/192M [00:10<00:00, 19.0MB/s]

Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/victorolufemi/spot-the-mask-challenge/versions/1

```
[ ]: import shutil

target_dir = "/content"
shutil.copytree(path, target_dir, dirs_exist_ok=True)

print("Copied dataset to:", target_dir)
```

Copied dataset to: /content

The data comes in a zip file. This command extracts it into the 'images' folder. The '-q' option stops it listing all the files and keeps our notebook looking tidy.

```
[ ]: # Extracting the files
# !unzip -q images.zip # My file is not zipped so it's all good fam
```

Normally, Colab is great in that everything you need to get started is present. However, we want our libraries updated, and to make sure we have the latest (best) of our major libraries. Below, we install FastAI and Timm.

4 Installing Libraries

4.0.1 About FastAI

FastAI is an open-source deep learning library built on top of PyTorch that provides a high-level API for researchers and practitioners to build, train, and deploy machine learning models efficiently, particularly for computer vision, natural language processing, and tabular data analysis. It includes pre-trained models, data augmentation tools, and support for large datasets and distributed training.

Before you grin, I really love this library because it can serve as a very high level one where many things look abstracted and it can go very very low level where you can do lots of crazy stuff if you know what you are doing.

4.0.2 About Timm

The Timm library is a popular open-source computer vision library built on top of PyTorch. It is created by Ross Wightman contains collection of SOTA computer vision models, layers, utilities,

optimizers, schedulers, data-loaders, augmentations and also training/validating scripts with ability to reproduce ImageNet training results.

```
[ ]: %%capture
      !pip install timm
      !pip install fastai
```

With installations out the way, we can load in some libraries. It's good practice to keep all your imports right at the top. The most important ones here are pandas (for making it easy to work with tabular data) and fastai (for the actual deep learning).

```
[ ]: import pandas as pd
      import timm
      import warnings
      from fastai.vision.all import * #import everthing from vision
      warnings.filterwarnings('ignore')
```

Import FastAI to check out your version to avoid disparities. As at the time of writing mine was 2.8.6.

```
[ ]: import fastai
      print(fastai.__version__ )
```

2.7.19

5 Taking a look at the data

The labels come as a CSV file. For convenience and to check out the data, we can load it into a dataframe using pandas.

If you are getting an error, it means you've not uploaded the CSV files. Download the CSV files from [here](#) and upload them through the left tab of this notebook. Or check the code below the kaggle one to be sure thing run smoothly.

```
[ ]: train_df = pd.read_csv("train_labels.csv")
      ss = pd.read_csv("SampleSubmission.csv")
```

```
[ ]: train_df.tail()
```

```
[ ]:
      image  target
1303  hxjwafskxmlfaotwaklzuwuccsbxfu.jpg      0
1304  oyodauphffgmzmvlqlykcfvmxartok.jpg      1
1305  uppvtemhkneqggtbfubeccqjvyefiw.png      1
1306  mzjjvzkzvqmyukzsegtoktaslejcdz.jpg      0
1307  dbjdjyhnbvblwhhnfzaahzhdezbbib.jpg      0
```

Set Seed.

We definitely don't want to be in the shoes of those not able to reproduce their score, so let's seed everything seedable....

```
[ ]: def random_seed(seed_value, use_cuda):
    np.random.seed(seed_value)
    #cpu vars
    torch.manual_seed(seed_value)
    # cpu vars
    random.seed(seed_value)
    # Python
    if use_cuda:
        torch.cuda.manual_seed(seed_value)
        torch.cuda.manual_seed_all(seed_value)
    # gpu vars
    torch.backends.cudnn.deterministic = True
    #needed
    torch.backends.cudnn.benchmark = False
    #Remember to use num_workers=0 when creating the DataBunch.
```

```
[ ]: # You can use almost any integer (less than the number of random generators on
    ↪your pc)
    random_seed(2026, True) #Let's use your fav year
```

preview DataFrames

```
[ ]: train_df.head()
```

```
[ ]:
           image  target
0  tfymlmdkpzkqjdjhdxyhnoeuqsxzphw.jpg          0
1  rxgismgsvmaayzjarbfjaljhqmpbrt.jpg          1
2  uuzshfrhkgrkolhwdvliqauzulurnz.jpg          0
3  mjspxsagzusaznvnyxgamtrlqkqklp.jpg          0
4  rlbmuajgezfidjzlyeoupxpqbkbpt.jpg          1
```

5.1 Getting the data ready for the model

Creating your Data block

Remember to checkout the official Fastai [website](#) and [docs](#) to learn more about FastAI.

Creating a datablock is quite easy, the Datablock API is the foundational block of the dataset creation, you'll have to tell it you are creating a dataset of images, and you'll be using it for a classification task. Hence the Images and category block reference used.

The splitter simply splits your train set into train and validation, Fastai is very keen on overfitting and validation, even if you don't set the splitter, it does that automatically for you using defaults. Personally I prefer the Train_test splitter. It helps me take advantage of the stratification unlike random splits.

in case you are wondering, stratification means we are able to split based on class or label.

Next, you'll have to tell the API how to fetch the location of the images, and the respective label for that image. Remember that I've put all these stuff in a nice dataframe

Finally, you can do some transformations for the images. Remember that to train a Convolutional neural network, images are best fitted in when they are of the same size. to this effect, I've told Fastai to squish (not crop) all images to 320 by 320.

When you are done, create an instance of the datablock called the dataloader.

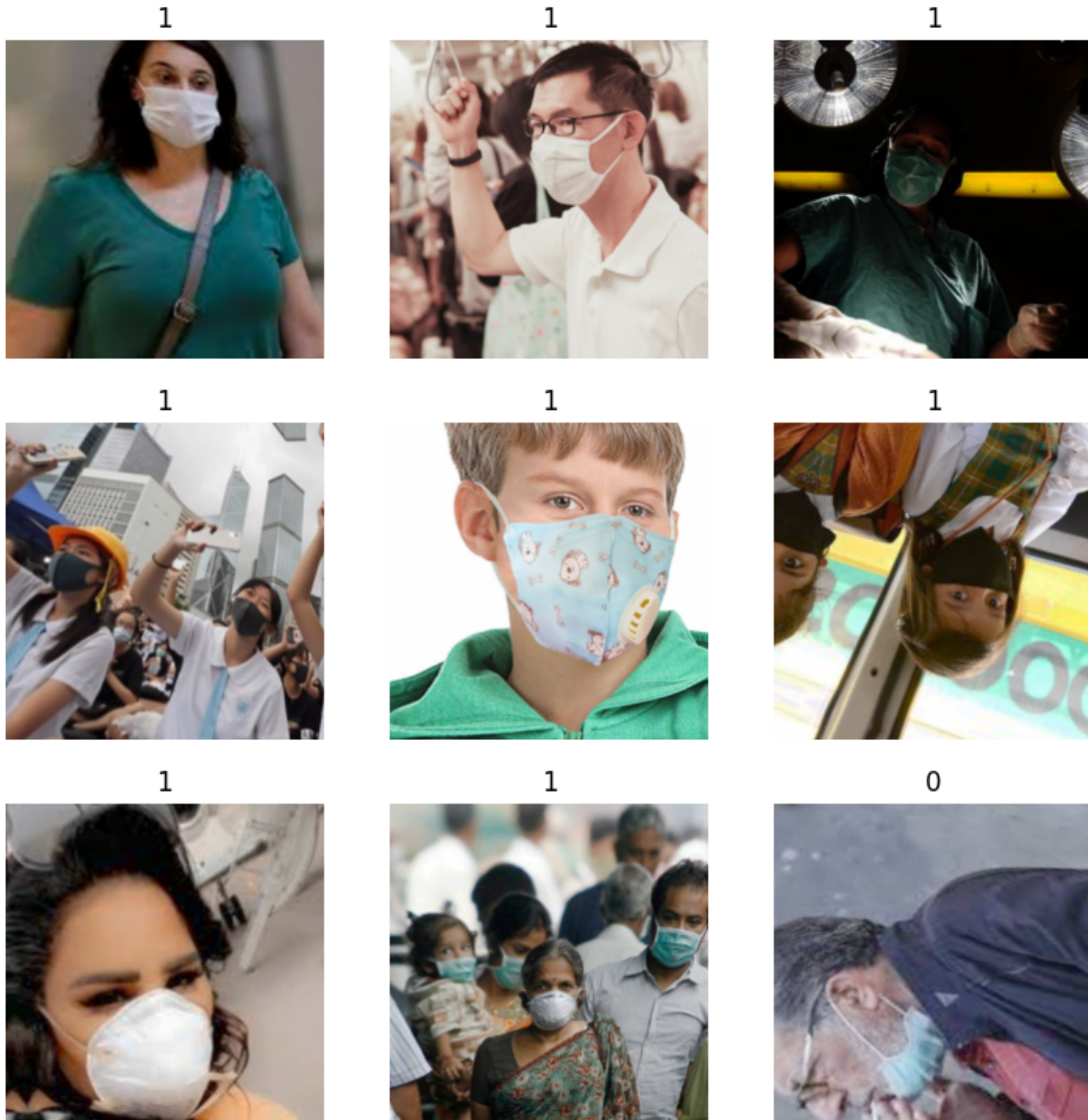
```
[ ]: Nosemask = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    splitter=TrainTestSplitter(0.1, stratify=train_df["target"]),
    get_x = ColReader(0, pref = "/content/images/images/" ),
    get_y=ColReader(1),
    item_tfms = Resize(460),
    batch_tfms = □
    ↪aug_transforms(do_flip=True,flip_vert=True,max_lighting=0.4,max_zoom=1.
    ↪2,max_warp=0.2,max_rotate=30,xtra_tfms=None)
)
```

Creating the Dataloader

```
[ ]: dls = Nosemask.dataloaders(train_df, bs=32, num_workers=0)
```

Check out one batch

```
[ ]: dls.show_batch()
```



Deep Dive into Data Augmentation. Data augmentation is a technique used in deep learning to increase the size of the training dataset by applying various transformations to the original images. This can help to reduce overfitting and improve the generalization performance of the model.

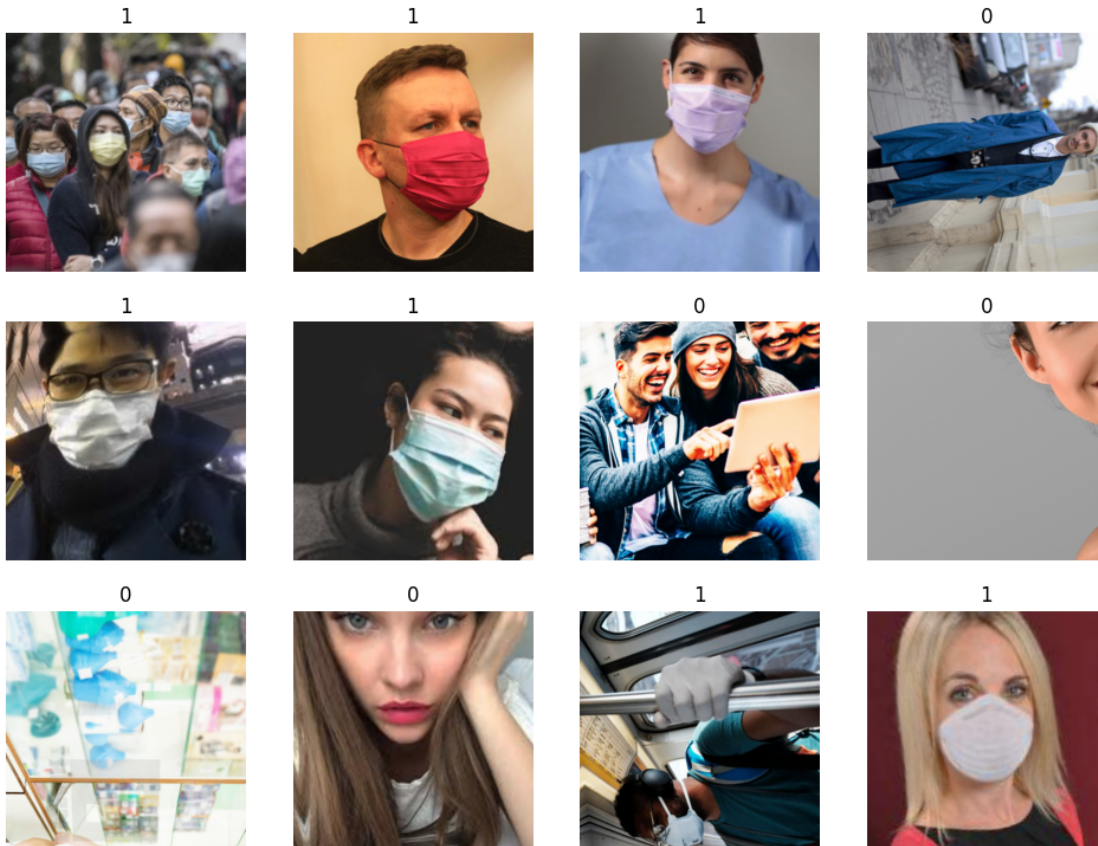
For image data, data augmentation techniques include rotation, scaling, flipping, cropping, and changing the brightness, contrast, and hue of the images. By applying these transformations to the training data, the model can learn to recognize objects and patterns from a wider range of perspectives and lighting conditions.

Data augmentation can be implemented using various libraries and frameworks, not limited to FastAI and PyTorch, which provide pre-defined functions for applying common transformations.

In FastAI you can customize by defining the `item_tfms` and the `batch_tfms`

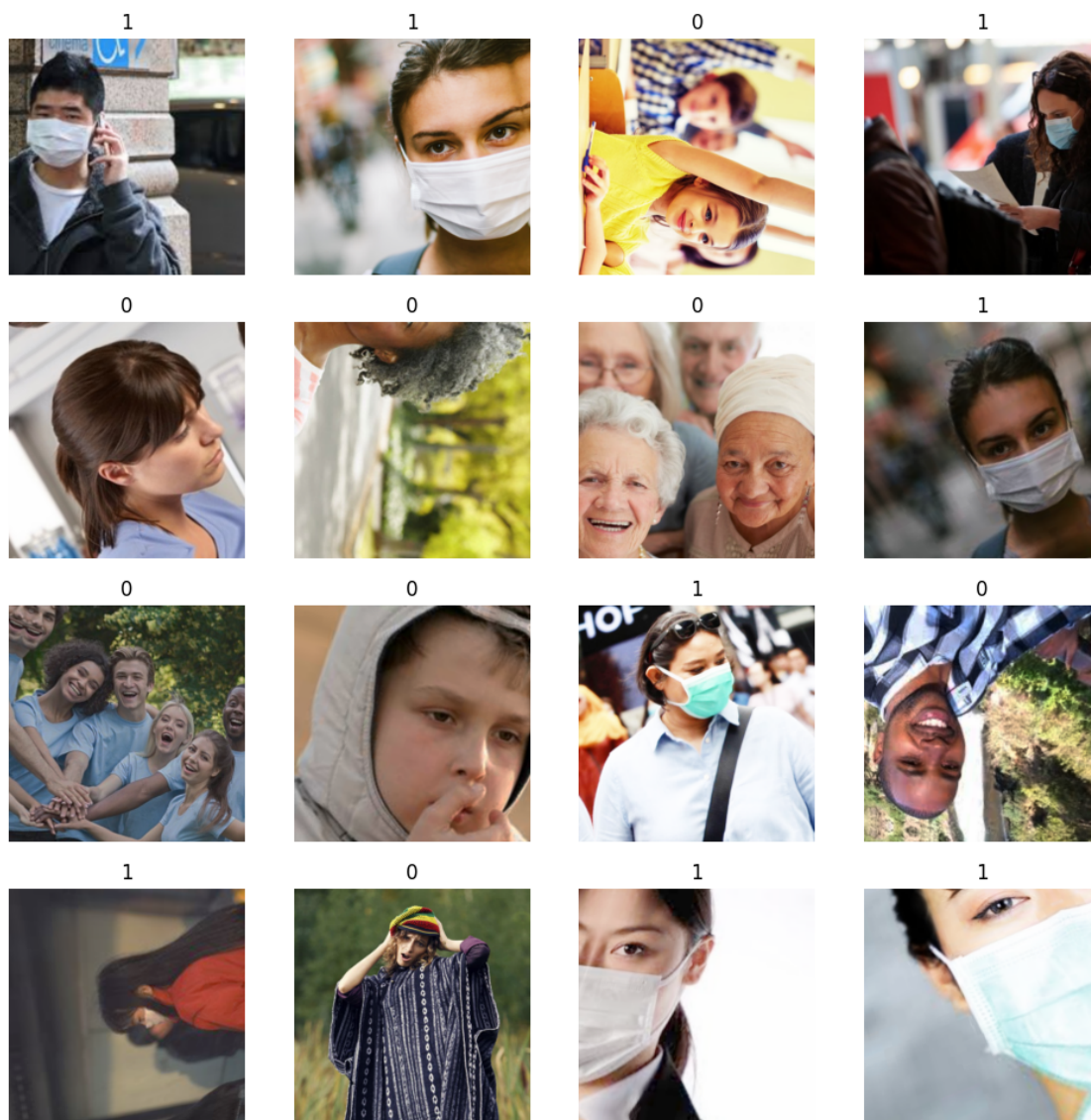
Example 1: Padding Images

```
[ ]: dls = dls.new(item_tfms=Resize(512, ResizeMethod.Pad, pad_mode='zeros'),  
    ↪ batch_tfms = aug_transforms(do_flip=True, flip_vert=True, max_lighting=0.4))  
dls.show_batch(max_n=12, nrows=3)
```



Example 2: Squishing Images

```
[ ]: ## squish the images  
  
dls = dls.new(item_tfms=Resize(128, ResizeMethod.Squish), batch_tfms =  
    ↪ aug_transforms())  
dls.show_batch(max_n=16, nrows=4)
```

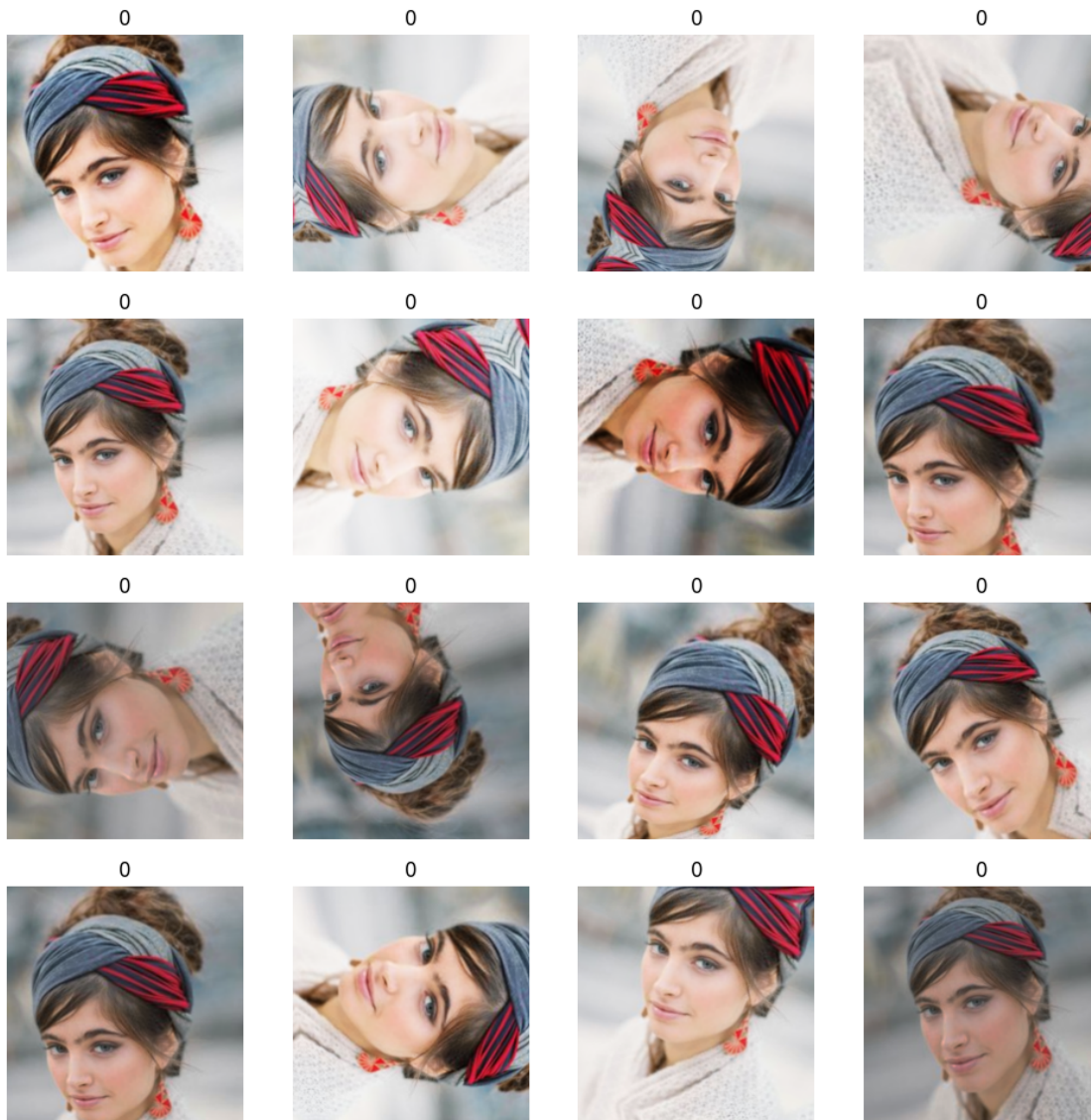
Example 3: Random Cropping: Random Resized Cropping

```
[ ]: ## random cropping
dls = dls.new(item_tfms=RandomResizedCrop(128,min_scale=0.3))
dls.show_batch(max_n=16, nrows=4)
```




Example 4 : Looking at a single image

```
[ ]: dls = dls.new(item_tfms=RandomResizedCrop(128,min_scale=0.3))
     dls.show_batch(max_n=16, nrows=4, unique=True)
```



Example 5: Multiplying defaults (increasing variety and complexity)

```
[ ]: dls = dls.new(items_tfms=Resize(128),batch_tfms=aug_transforms(mult=4))
      dls.show_batch(max_n=16, nrows=4)
```



```
[ ]: dls = Nosemask.dataloaders(train_df, bs=32, num_workers=0) #Let's use ours,
      ↪ stay with definitions in databunch
```

Explore Timm You can list the available models in the Timm library using the `list_models()` function. This function is provided by the Timm library and can be used to generate a list of all the available model names. You can also filter models by passing a wild card filter. For instance `res*` returns all the models that start with “res”

```
[ ]: import timm
      timm.list_models('res*')
```



```
[ ]: ['res2net50_14w_8s',
      'res2net50_26w_4s',
      'res2net50_26w_6s',
      'res2net50_26w_8s',
      'res2net50_48w_2s',
      'res2net50d',
      'res2net101_26w_4s',
      'res2net101d',
      'res2next50',
      'resmlp_12_224',
      'resmlp_24_224',
      'resmlp_36_224',
      'resmlp_big_24_224',
      'resnest14d',
      'resnest26d',
      'resnest50d',
      'resnest50d_1s4x24d',
      'resnest50d_4s2x40d',
      'resnest101e',
      'resnest200e',
      'resnest269e',
      'resnet10t',
      'resnet14t',
      'resnet18',
      'resnet18d',
      'resnet26',
      'resnet26d',
      'resnet26t',
      'resnet32ts',
      'resnet33ts',
      'resnet34',
      'resnet34d',
      'resnet50',
      'resnet50_clip',
      'resnet50_clip_gap',
      'resnet50_gn',
      'resnet50_mlp',
      'resnet50c',
      'resnet50d',
      'resnet50s',
      'resnet50t',
      'resnet50x4_clip',
      'resnet50x4_clip_gap',
      'resnet50x16_clip',
      'resnet50x16_clip_gap',
      'resnet50x64_clip',
      'resnet50x64_clip_gap',
```

'resnet51q',
'resnet61q',
'resnet101',
'resnet101_clip',
'resnet101_clip_gap',
'resnet101c',
'resnet101d',
'resnet101s',
'resnet152',
'resnet152c',
'resnet152d',
'resnet152s',
'resnet200',
'resnet200d',
'resnetaa34d',
'resnetaa50',
'resnetaa50d',
'resnetaa101d',
'resnetblur18',
'resnetblur50',
'resnetblur50d',
'resnetblur101d',
'resnetrs50',
'resnetrs101',
'resnetrs152',
'resnetrs200',
'resnetrs270',
'resnetrs350',
'resnetrs420',
'resnetv2_18',
'resnetv2_18d',
'resnetv2_34',
'resnetv2_34d',
'resnetv2_50',
'resnetv2_50d',
'resnetv2_50d_evos',
'resnetv2_50d_frn',
'resnetv2_50d_gn',
'resnetv2_50t',
'resnetv2_50x1_bit',
'resnetv2_50x3_bit',
'resnetv2_101',
'resnetv2_101d',
'resnetv2_101x1_bit',
'resnetv2_101x3_bit',
'resnetv2_152',
'resnetv2_152d',

```
'resnetv2_152x2_bit',
'resnetv2_152x4_bit',
'resnext26ts',
'resnext50_32x4d',
'resnext50d_32x4d',
'resnext101_32x4d',
'resnext101_32x8d',
'resnext101_32x16d',
'resnext101_32x32d',
'resnext101_64x4d']
```

6 Time for training!

Now it's time to train a model. We're going to take advantage of transfer learning, and start with a nice pretrained model. There are many to choose from as listed above in the [timm library](#). Here, we'll use a ResNet18.

There are fancier models that might take longer to train but can give better accuracy - check out the minimal example at the end of this notebook to see another model in action.

If you want to know which models are the best, check out this notebook from Jeremy Howard, the founder of FastAI. <https://www.kaggle.com/code/jhoward/which-image-models-are-best/>

Creating a 'learner' with our chosen model architecture is as simple as:

```
[ ]: learn = vision_learner(dls, 'resnet18', metrics=[accuracy])
```

```
model.safetensors:  0%|          | 0.00/46.8M [00:00<?, ?B/s]
```

We've passed in accuracy as a metric, out of interest. The 'valid_loss' displayed during training is the log loss, and should closely match your score on Zindi.

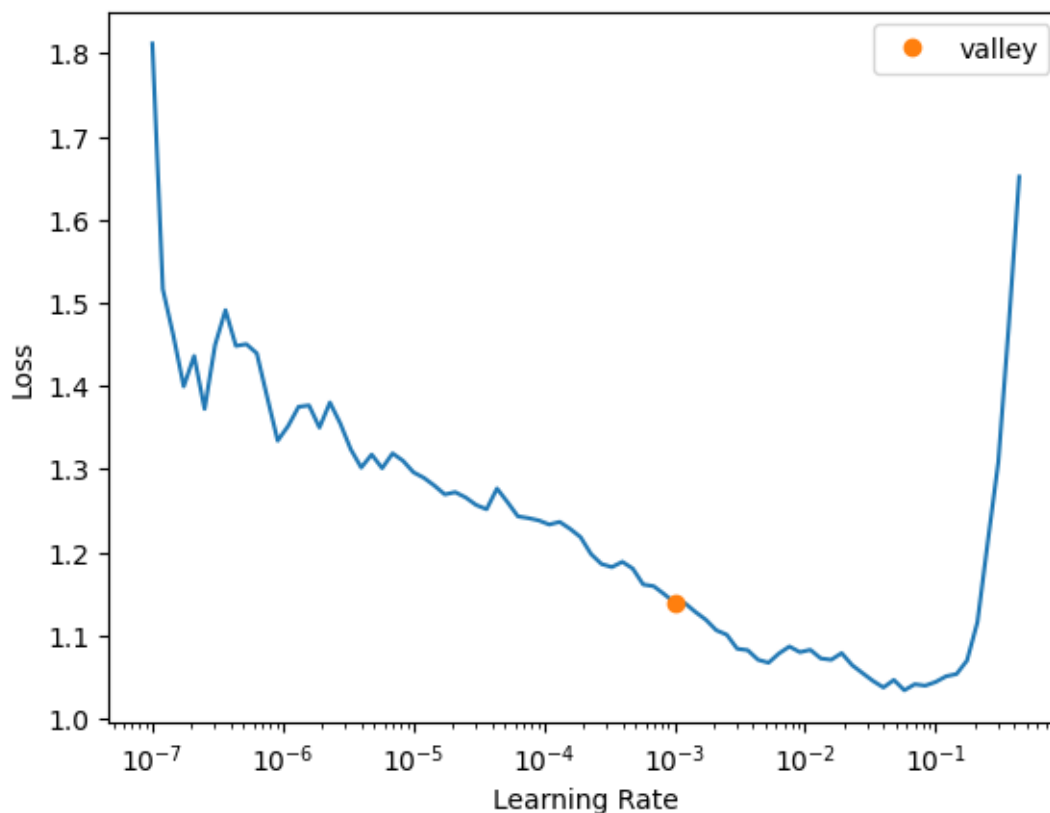
We could start training, but first let's check out what changing the learning rate looks like.

```
[ ]: learn.lr_find()
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
[ ]: SuggestedLRs(valley=0.0010000000474974513)
```

```
[ ]: #learner.fine_tune(3, base_lr=0.002511) #use your own base learning rate
      learn.fine_tune(3) #use your own base learning rate
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

We're already at 97% accuracy! Amazing.

In FastAI, the `learner.fine_tune()` method is used to fine-tune a pre-trained neural network on a new dataset. Fine-tuning involves updating the weights of the pre-trained network using the new dataset while retaining the learned knowledge from the original dataset.

The `learner.fine_tune()` method automatically sets the learning rate and schedules it for training. By default, it performs one cycle of discriminative fine-tuning, where the last few layers of the network are trained at a much higher learning rate than the rest of the layers.

you can read more on the docs: <https://docs.fast.ai/tutorial.vision.html>

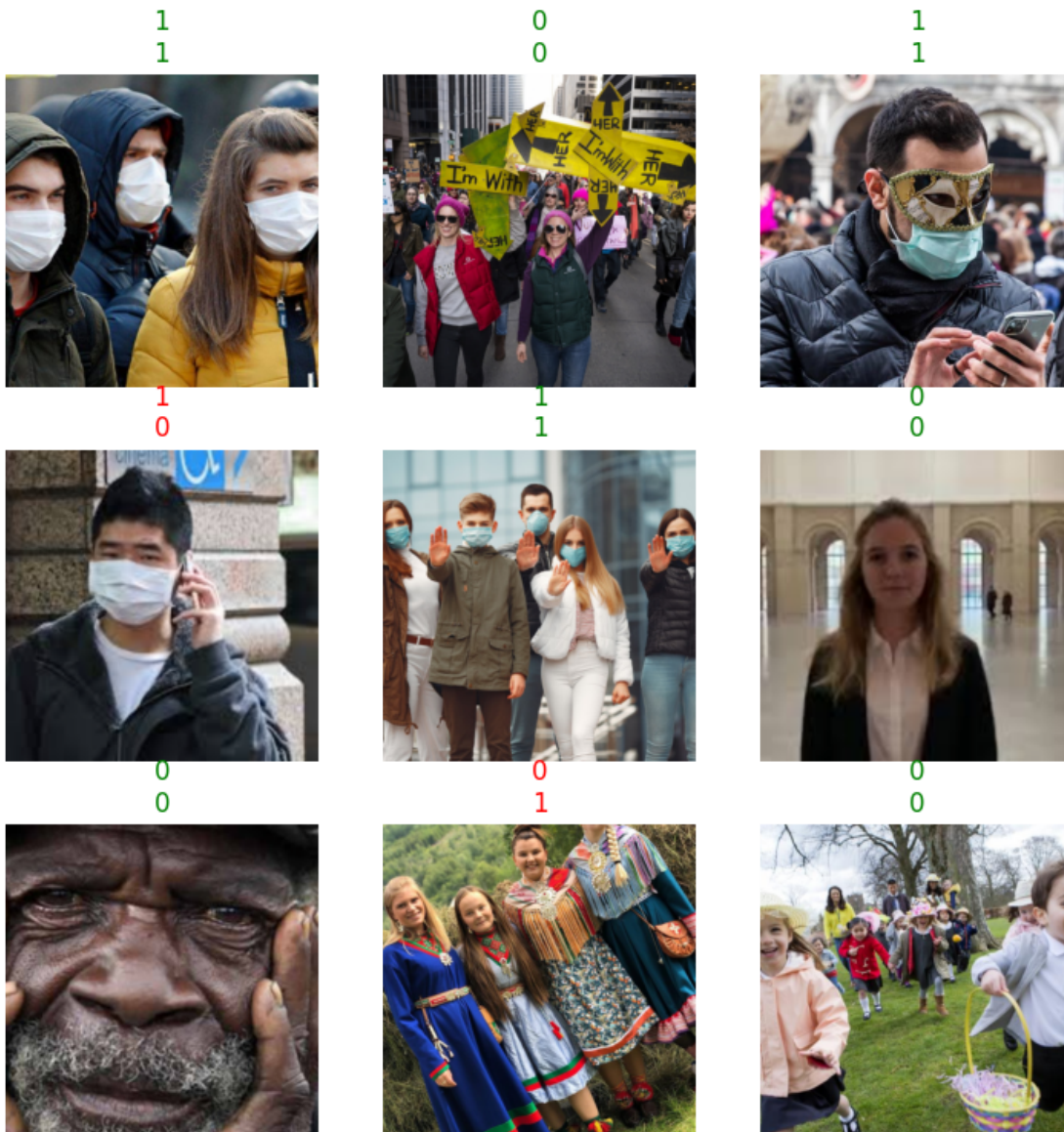
Check the results

In FastAI, the `learn.show_results()` method is used to visualize the performance of a trained model on the validation dataset.

```
[ ]: learn.show_results()
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```



Check the Confusion Matrix

A confusion matrix is a useful tool for evaluating the performance of a classification model, as it provides more detailed information than a simple accuracy score. It can help identify classes where

the model is making more errors, such as misclassifying one class as another.

In FastAI, you can generate a confusion matrix for a trained model using the `interp.plot_confusion_matrix()` method. This method takes the validation dataset as input and generates a graphical representation of the confusion matrix.

```
[ ]: interp = ClassificationInterpretation.from_learner(learn)

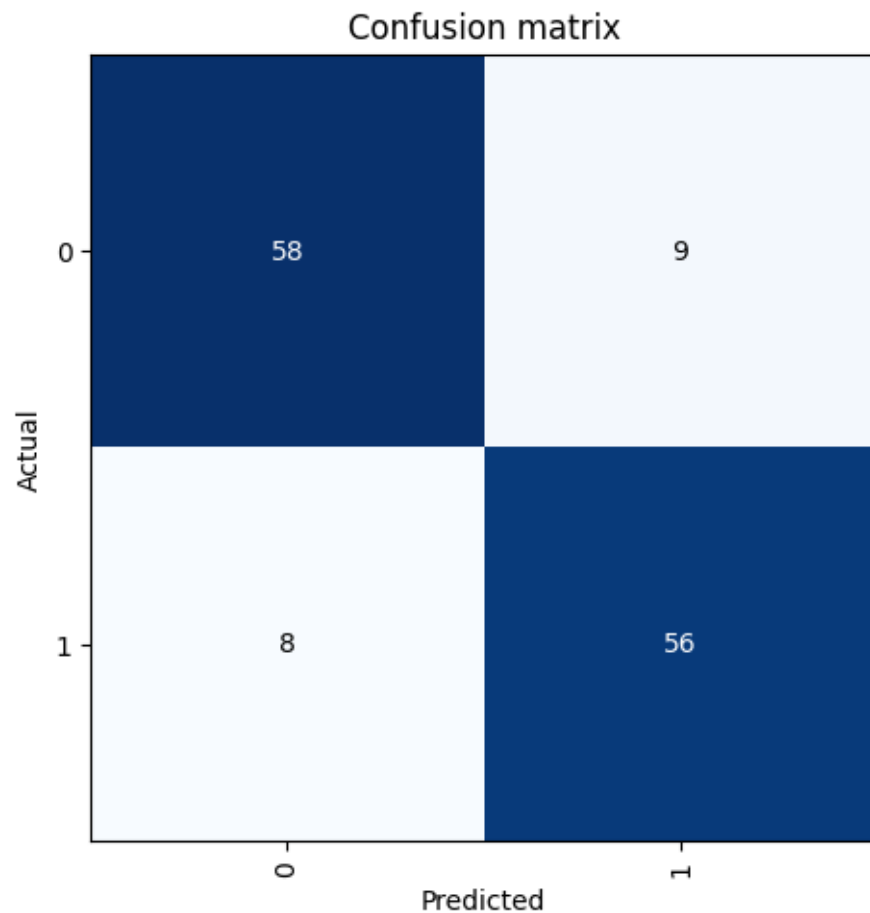
interp.plot_confusion_matrix()
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>



Plot the Top Losses

In FastAI, `learn.plot_top_losses()` is a method that allows you to visualize the samples in a validation set that the model had the most trouble predicting.

When called, `learn.plot_top_losses()` will display a grid of images from the validation set that were most misclassified by the model. These images are typically sorted by the model's loss values, with the highest loss images appearing at the top of the grid.

```
[ ]: ## plot_top_losses which shows us the images with the highest loss in our  
    dataset
```

```
interp.plot_top_losses(5)
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Prediction/Actual/Loss/Probability

0/1 / 5.09 / 0.99



0/1 / 4.98 / 0.99



0/1 / 3.14 / 0.96



1/0 / 2.90 / 0.95



1/0 / 2.76 / 0.94



7 Making Predictions

The sample submission file lists the test images, and shows a target column of all 0s. We want to replace those 0s with our predicted targets - close to 1 for mask, close to 0 for no mask. We'll be scored on how close we are to the true values, but if we simply put 1s and 0s then any mistakes will result in a bad score. Instead, we'll use the model's raw outputs instead. That way, when the model is confident, we predict close to 1. But when it's unsure, we predict something lower (say 0.8, or 0.2 if it's probably no mask) and if it's wrong we won't get such a large penalty.

```
[ ]: ss.tail() #see what the test set looks like
```

```
[ ]:
      image target
504  zpfhbgixcctxylnihhrepjgcmjksvf.jpg      0
505  zuignwckjyxefmpubjpefjvhghaoi.jpg      0
506  zuxvwdblfwjpijbexfglpyreqslhs.jpg      0
507  zvyajojutzgqumdjfxkpiuuxeefyy.jpg      0
508  zynasgxnznzqaallnvkthegctommtu.jpg      0
```

To make predictions, we need to create the test dataset in a format that fastAI can take in, we do this like so:

```
[ ]: tdl = dls.test_dl(ss) # quickly create test data loader
```

Now we can get predictions:

```
[ ]: test_preds_, test_labels_tta = learn.get_preds(dl=tdl) #Get Preds
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
[ ]: # What do these look like?
      test_preds_[0] # The first prediction
```

```
[ ]: tensor([0.9127, 0.0873])
```

The predictions take the form [P(no_mask), P(mask)]. We want the second one, and we can add it to our target column like so:

```
[ ]: train_df.tail()
```

```
[ ]:
      image target
1303  hxjwafskxmlfaotwaklzuwuccsbxfu.jpg      0
1304  oyodauphffgmzmvqlykcfvmxartok.jpg      1
1305  uppvtemhkneqggtbfubeccqjvyefiw.png      1
1306  mzjjvzkzvqmyukzsegtoktaslejc dz.jpg      0
1307  dbjdjyhnbvblwhhnfzaahzhdezbbib.jpg      0
```

Preparing your Submission

```
[ ]: submission = ss.copy()
      submission.head()
```

```
[ ]:
      image target
0  aadawlxbmapqrbglxyzarhjasgiobu.png      0
1  abpxvdfyhaaohzcrngcucmhffwizxs.jpg      0
2  aclkjfvackiieiznzfcwienplielrj.jpg      0
3  aelkivmayxgsdjosiibfgmkvfrjvjk.jpg      0
4  aelzzshpfxerelefnkatpczktuxjln.jpg      0
```



```
[ ]: submission['target'] = [float(p[1]) for p in test_preds_] # we want p(Mask) for
    our submission, so we'll look at the second number in each prediction with
    p[1]
    submission.head()
```

```
[ ]:
      image      target
0  aadawlxbmapqrbldgxyzarhjasgiobu.png  0.087300
1  abpxvdfyhahzcrngcucmhffwizxs.jpg    0.004029
2  aclkjfvackiieiznzcwienplielrj.jpg    0.998556
3  aelkivmayxgsdjosiibfgmkvfrjvjk.jpg    0.923519
4  aelzzshpfxerelefnkatpczktuxjln.jpg    0.000271
```

```
[ ]: # Save the file
    submission.to_csv('starter_nb_baseline.csv', index=False)
```

You can submit to Zindi to see how well this does!

8 Making it better

There are some easy ways to boost your score. - Use test time augmentation (tta) - Try a fancier model - Train more (not always better, but training for a few more epochs will probably help) - Combine predictions from several models.

Let's try a few of these

TTA

```
[ ]: test_preds_tta, test_labels_tta = learn.tta(dl=tdl) #TTA
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```
[ ]: submission['target'] = [float(p[1]) for p in test_preds_tta]
    submission.head()
```

```
[ ]:
      image      target
0  aadawlxbmapqrbldgxyzarhjasgiobu.png  0.201200
1  abpxvdfyhahzcrngcucmhffwizxs.jpg    0.025573
2  aclkjfvackiieiznzcwienplielrj.jpg    0.992136
3  aelkivmayxgsdjosiibfgmkvfrjvjk.jpg    0.669861
4  aelzzshpfxerelefnkatpczktuxjln.jpg    0.004257
```

```
[ ]: # Save the file
    submission.to_csv('starter_nb_baseline_with_TTA.csv', index=False)
```

It takes longer, since it's making predictions on 8 variations of each test image. But TTA usually gives a better score. Did it work for you?

Now for combining predictions. There are many ways of doing this, but let's take the easy route and assume you've got several CSV files with previous submissions. Here, we load some in and take the average. Of course, we should really combine predictions from DIFFERENT models, but this is just for illustration. It's up to you to train a few more models and combine :)

```
[ ]: # Average of two sets of predictions
preds1 = pd.read_csv('starter_nb_baseline.csv')
preds2 = pd.read_csv('starter_nb_baseline_with_TTA.csv')
av_preds = pd.DataFrame({
    'image': preds1['image'],
    'target': 0.5*preds1['target'] + 0.5*preds2['target']
})
av_preds.to_csv('ensemble1.csv', index=False)

# Exercise: Run the minimal example below and combine those predictions with
↳ the tta submission from above

# Exercise: If one model is substantially better, we may want it's predictions
↳ to count more towards the final prediction.
# Try to use 0.8*good_predictions + 0.2*other_predictions to skew your
↳ ensemble predictions towards the better model's outputs.

[ ]: exit() # A colab function that restarts notebook
```

9 Minimal Example - 10 Lines of Code

We've got a lot of explanation in here, but here is a minimal submission from scratch in only 10 lines of code. You'll notice we're using a fancier model, the Convnext tiny model from Timm. Is it better? You'll have to test and find out.

We're also adding `.mixup()` since that sounds fancy, and training for longer than the example. Feel free to play with this, and see how good you can get it!

You can read about mixup and friends [here](#)

Good luck :)

```
[ ]: from fastai.vision.all import * #import everything from vision
import pandas as pd
import timm

[ ]: train_df = pd.read_csv("train_labels.csv")
submission = pd.read_csv("SampleSubmission.csv")
```

```

Nosemask = DataBlock(blocks=(ImageBlock, CategoryBlock),
↳splitter=TrainTestSplitter(0.1, stratify=train_df["target"]),get_x =
↳ColReader(0, pref = "/content/images/images/" ),get_y=ColReader(1),item_tfms=
↳Resize(460), batch_tfms =
↳aug_transforms(do_flip=True,flip_vert=True,max_lighting=0.4,max_zoom=1.
↳2,max_warp=0.2,max_rotate=30,xtra_tfms=None))
dls = Nosemask.dataloaders(train_df, bs=16, num_workers=0)
learn = vision_learner(dls,'convnext_tiny', metrics=[accuracy], path=".") #try
↳convnext_base and convnext_large
learn.fine_tune(3,cbs=MixUp) #Apply Mixup #increase no of Epochs
tdl = dls.test_dl(submission) # quickly create test data loader
test_preds_, test_labels_ = learn.get_preds(dl=tdl) #Get Preds
submission['target'] = [float(p[1]) for p in test_preds_]
submission.to_csv('convnext_tiny_model.csv', index=False)

```

```

/usr/local/lib/python3.11/dist-packages/fastai/data/transforms.py:214:
FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
    o = r[c] if isinstance(c, int) or not c in getattr(r, '_fields', []) else
getattr(r, c)

```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

9.1 Export model

The `learn.export()` method is used to export a trained model so that it can be used for inference or deployment in other applications.

`learn.export()` will save the model's architecture, weights, and any custom transformations or pre-processing steps to a file. The resulting file can be loaded into a new instance of the Learner class using the `load_learner()` method, allowing the model to be used for inference on new data.

This method is useful for sharing trained models with others or for deploying models to production environments. By exporting a trained model, you can ensure that others can use your model without needing to retrain it from scratch.

```

[ ]: #will save an export file called a model.pkl
learn.export('model.pkl')

```

10 Deployment example

Model deployment refers to the process of making a trained machine learning model available for use in a production environment or application.

It typically involves taking the trained model and integrating it into a larger software system, such as a web application, mobile app, or data processing pipeline. This can involve creating APIs or other interfaces for the model, integrating it with databases or other data sources, and configuring the model to run in a specific runtime environment.

The goal of model deployment is to make the trained model available for use by others, either within an organization or to external users.

Effective model deployment actually requires careful planning and coordination between data scientists, software developers, and IT professionals, as well as a deep understanding of the target environment and use cases for the model.

In the below example I'll show you how we can use an exported fastAI model

Remember that we exported our model after we trained it much earlier, we saved it as “model.pkl”. Now we retrieve it using the `load_learner()` function in FastAI.

We'll call this “new_learner”

```
[ ]: new_learner=load_learner("model.pkl")
```

```
/usr/local/lib/python3.11/dist-packages/fastai/learner.py:455: UserWarning:
load_learner` uses Python's insecure pickle module, which can execute malicious
arbitrary code when loading. Only load files you trust.
If you only need to load model weights and optimizer state, use the safe
`Learner.load` instead.
```

```
warn("load_learner` uses Python's insecure pickle module, which can execute
malicious arbitrary code when loading. Only load files you trust.\nIf you only
need to load model weights and optimizer state, use the safe `Learner.load`
instead.")
```

use wget to download an image from the internet, you can try this with any image you have by uploading it to the files tab

```
[ ]: %%capture
!wget "https://covidsafeuk.com/wp-content/uploads/2020/11/1605627334852.jpeg"
```

Making inference is as simple as `learner.predict()`

```
[ ]: new_learner.predict("/content/1605627334852.jpeg")
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
[ ]: ('1', tensor(1), tensor([1.1532e-04, 9.9988e-01]))
```

Our Model says “1”, meaning the image has a mask on, let's check if it truly does.

```
[ ]: im = Image.open('/content/1605627334852.jpeg') #open image
plt.imshow(im) #show image
```

```
[ ]: <matplotlib.image.AxesImage at 0x7b4f92b2d350>
```



Amazing! We've just created and tested our own A.I that can predict if someone is wearing a mask or not.

Check no-mask image

```
[ ]: # download another image from the internet with no mask
%%capture
!wget "https://engineering.cmu.edu/_files/images/directory/jbarros.png"
```

Make Prediction

```
[ ]: new_learner.predict("/content/jbarros.png")
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```
[ ]: ('0', tensor(0), tensor([0.9923, 0.0077]))
```

A.I model says "0" meaning no-mask. Let's confirm this


```
[ ]: im = Image.open('/content/jbarros.png')
plt.imshow(im)
```

```
[ ]: <matplotlib.image.AxesImage at 0x7b4f863bda10>
```



Congratulations on Finishing up the Tutorial walk through Building a model to testing and to single inference.

- Keep Learning
- Keep Winning

I am always open to assist enthusiasts with difficulties they are facing in Machine learning and Deep learning. Feel free to reach out to me:

. [Twitter](#).

. [Linkedin](#).

. [Github](#)

Author: Olufemi Victor Tolulope.

```
[ ]:
```